SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

FACULTY OF SCIENCE AND HUMANITIES

DEPARTMENT OF COMPUTER APPLICATIONS



PRACTICAL RECORD NOTE

STUDENT NAME :

REGISTER NUMBER :

CLASS : MCA

SECTION :

YEAR & SEMESTER : II YEAR & III SEM

SUBJECT CODE : PCA25D08J

SUBJECT TITLE : AUGMENTED REALITY AND VIRTUAL REALITY

OCTOBER 2026

SRM
INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

FACULTY OF SCIENCE AND HUMANITIES

DEPARTMENT OF COMPUTER APPLICATIONS

SRM Nagar, Kattankulathur – 603 203

CERTIFICATE

Certified to be the bonafide record of practical work done by

_____

*Register No.*_____ *of* **MCA** *Degree Programme for* ***PCA25D08J - AUGMENTED REALITY AND VIRTUAL REALITY*** *in the Computer lab in SRM Institute of Science and Technology during the academic year 2026-2027.*

Staff In-charge *Head of the Department*

*Submitted for Semester Practical Examination held on*_____.

Internal Examiner *External Examiner*

INDEX

S.NO	DATE	LIST OF PROGRAMS	PAGE NO	STAFF SIGN
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				

EXERCISE 1

DATE :

Implementation on Video

Aim:

To implement a video/feature viewing application in Virtual Reality (VR) using Unity, where the user can view and interact with a video or selected features in a 3D environment..

Procedure

Step 1: Create a New Unity Project

1. Open Unity Hub.
2. Click New Project.
3. Select the 3D Core template.
4. Enter the project name as Video Feature Viewing.
5. Click Create Project.

Step 2: Set Up the VR Environment

1. Go to Edit → Project Settings.
2. Select XR Plug-in Management.
3. Install XR Plug-in Management if it is not already installed.
4. Enable the appropriate XR plug-in for the target VR device.
5. Add an XR Origin or VR camera setup to the scene.

Step 3: Create the Video Environment

1. Create a Plane or suitable 3D surface in the scene.
2. Position the surface in front of the VR camera.
3. Add a Video Player component.
4. Import the required video file into the Assets folder.
5. Assign the video to the Video Player component.

Step 4: Configure Video Playback

1. Set the Video Player to play the selected video.
2. Configure the video to display on the created plane.
3. Adjust the plane size and position for comfortable viewing.
4. Set the required playback options such as Play on Awake or manual playback.

Step 5: Add Viewing Features

1. Add UI buttons for Play, Pause, Stop, and Replay.
2. Configure the buttons to control the video.
3. Add interaction features such as selecting or viewing different video features.
4. Test the interaction using the VR controller or mouse.

Step 6: Test the Application

1. Save the Unity scene.
2. Click Play in Unity.
3. View the video through the VR camera.
4. Test the available video controls and interactions.
5. Make necessary adjustments to the position, size, and playback settings.

Output

A VR-based video/feature viewing application is successfully implemented in Unity. The user can view the selected video in a 3D/VR environment and control its playback using interactive controls.



Result

The Video/Feature Viewing application was successfully implemented in Unity,

EXERCISE 2

DATE :

Implementation on Virtual Tour

Aim:

To implement a Virtual Tour application using Unity, allowing the user to explore a virtual environment and view different locations or scenes interactively.

Procedure

Step 1: Create a New Unity Project

1. Open Unity Hub.
2. Click New Project.
3. Select the 3D Core template.
4. Enter the project name as Virtual Tour.
5. Click Create Project.

Step 2: Create the Virtual Environment

1. Create a suitable 3D environment such as a museum, college, historical place, or building.
2. Add 3D objects such as rooms, walls, doors, furniture, and other required elements.
3. Arrange the objects to form the virtual location.
4. Add suitable lighting to the environment.

Step 3: Set Up the Camera

1. Add a Camera to the scene.
2. Position the camera at the starting point of the tour.
3. Configure the camera to provide a suitable first-person view.
4. For VR, configure the camera using the required XR Origin setup.

Step 4: Implement User Movement

1. Add a suitable player or XR Origin.
2. Configure movement controls using the keyboard, mouse, or VR controller.
3. Set the movement speed and viewing direction.
4. Test movement within the virtual environment.

Step 5: Create Tour Locations

1. Create different points or areas in the environment.
2. Add markers or interactive objects at important locations.
3. Provide information about each location using UI text or panels.
4. Allow the user to move between different areas of the virtual tour.

Step 6: Test the Virtual Tour

1. Save the scene.
2. Click Play in Unity.
3. Move through the virtual environment.
4. View the different locations and information panels.
5. Test the interaction and navigation features.

Output

A **3D virtual environment** is displayed, and the user can navigate through different locations, view objects, and interact with information points.



Result

The Virtual Tour application was successfully implemented in Unity, enabling the user to navigate through a virtual environment and explore different locations interactively.

EXERCISE 3

DATE :

Sweeping Coverage of Eye Movements

Aim:

To implement a sweeping eye-movement technique in a VR environment to observe and cover different areas of the virtual scene through systematic eye movement.

Procedure

Step 1: Create a New Unity Project

1. Open Unity Hub.
2. Click New Project.
3. Select the 3D Core template.
4. Enter the project name as Eye Movement Coverage.
5. Click Create Project.

Step 2: Create the VR Environment

1. Create a simple 3D environment using 3D objects.
2. Place objects at different positions around the user.
3. Add suitable lighting to the scene.
4. Add an XR Origin and configure the VR camera.

Step 3: Set Up Eye/View Tracking

1. Configure the VR camera to represent the user's viewing direction.
2. Enable the required head/eye tracking features supported by the VR device.
3. Position the objects at different viewing angles.
4. Set the required viewing range.

Step 4: Implement Sweeping Eye Movement

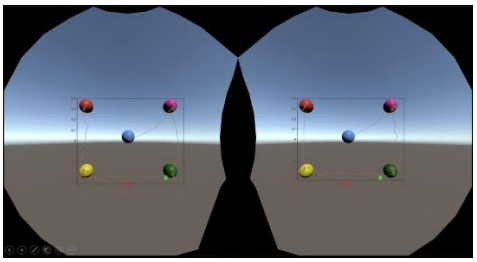
1. Run the VR application.
2. Ask the user to look from left to right across the scene.
3. Move the gaze gradually from right to left.
4. Repeat the movement from top to bottom and bottom to top.
5. Observe the different objects covered during the sweeping movement.

Step 5: Test the Application

1. Check whether objects placed at different positions are visible during the eye-movement sweep.
2. Repeat the sweeping movement several times.
3. Verify that the complete scene can be visually covered.
4. Record the observed eye/viewing movement if tracking data is available.

Output

The VR scene is displayed, and the user's gaze/view moves systematically across different regions of the virtual environment, covering objects placed at different positions**.**



Result

The sweeping coverage of eye movements was successfully implemented and observed in the VR environment

EXERCISE 4

DATE :

Implementation on Material Animation

Aim:

To implement material animation in Unity by animating the material properties of a 3D object to create a visually dynamic effect in an AR/VR environment.

Procedure

Step 1: Create a Unity Project

1. Open Unity Hub.
2. Click New Project.
3. Select the 3D Core template.
4. Enter the project name as Material Animation.
5. Click Create Project.

Step 2: Create a 3D Object

1. Go to **GameObject → 3D Object**.
2. Select **Cube** or **Sphere**.
3. Position the object in the scene.
4. Add suitable lighting to the scene.

Step 3: Create and Apply a Material

1. In the **Project** window, right-click and select **Create → Material**.
2. Give the material a suitable name, such as **AnimatedMaterial**.
3. Set the required material properties.
4. Drag the material onto the 3D object.

Step 4: Create Material Animation

1. Select the 3D object.
2. Open **Window → Animation → Animation**.
3. Click **Create** and save the animation clip.
4. Add material properties that need to be animated, such as **Color, Emission, or Texture Offset**.
5. Set different property values at different time points.
6. Unity creates keyframes for the selected material properties.

Step 5: Configure the Animation

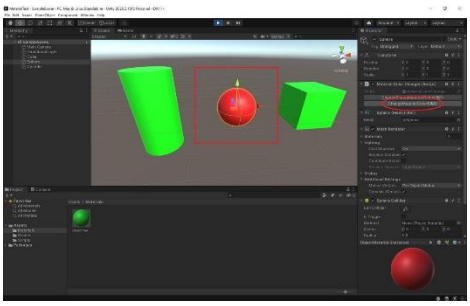
1. Adjust the animation duration.
2. Enable **Loop Time** if continuous animation is required.
3. Modify the keyframes to create a smooth material transition.
4. Save the scene.

Step 6: Test the Application

1. Click the **Play** button in Unity.
2. Observe the 3D object.
3. Verify that the material changes according to the animation.
4. Adjust the animation speed and material properties if required.

Output

The 3D object displays an animated material effect, such as changing color, glowing, or moving texture, based on the configured animation.



Result

The material animation was successfully implemented in Unity.

EXERCISE 5

DATE :

Audio and Visual effects

Aim

To experiment with different audio and visual effects in Unity in order to create an immersive user experience

Procedure

Step 1: Create a New Project

1. Open Unity Hub
2. Click **New Project**
3. Select **3D Core Template**
4. Click **Create**

Step 2: Create Basic Scen

1. In Hierarchy → Right-click → **3D Object → Plane** (Ground)
2. Add objects:
   Cube
   Sphere
3. Adjust positions in front of camera

PART A: ADD AUDIO EFFECTS

Step 3: Import Audio File

1. Download or use a sample audio file (.mp3/.wav)
2. Drag and drop it into **Assets**

Step 4: Add Audio Source

1. Select any object (e.g., Cube)
2. Click **Add Component → Audio Source**
3. Assign the audio clip
4. Enable:
   - **Play On Awake** (optional)
   - **Loop** (optional)

Step 5: Adjust Audio Settings

1. Modify:
   - Volume
   - Spatial Blend → Set to **3D**
2. Move around in scene to hear 3D sound effect

PART B: ADD VISUAL EFFECTS

Step 6: Add Light Effects

1. Right-click → **Light** → **Point Light**
2. Change:
    - Color
    - Intensity
    - Range

Step 7: Add Material Effects

1. Create material:
    - Right-click → **Create** → **Material**
2. Change:
    - Color
    - Metallic / Smoothness
3. Apply to objects

Step 8: Add Particle Effects

1. Right-click → **Effects** → **Particle System**
2. Modify:
    - Start Color
    - Size
    - Speed
3. Position near object

Step 9: Add Camera Effects

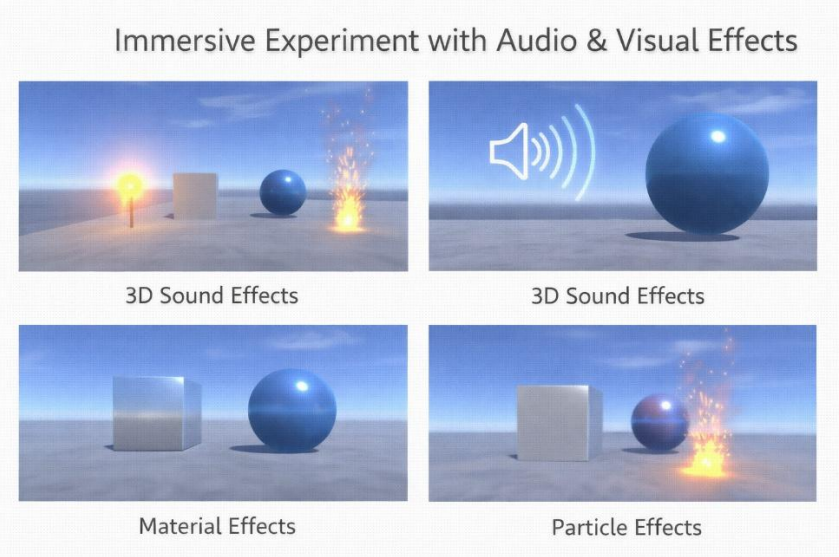
1. Select **Main Camera**
2. Add:
    - Post-processing (if available)
    - Adjust lighting and background

Step 10: Run the Scene

1. Click **Play**
2. Observe:
    - Audio plays in scene
    - Objects display lighting & particle effects

Output

- Audio plays with 3D spatial effect
- Objects show lighting, materials, and particle effects
- Scene appears more immersive



Result

Thus, different audio and visual effects were successfully implemented to create an immersive experience in Unity.

EXERCISE 6

DATE

Creation of Placement Indicator

Procedure

Step 1: Create a New Project

1. Open Unity Hub
2. Click **New Project**
3. Select **3D Core Template**
4. Click **Create**

Step 2: Create the Scene

1. In Hierarchy → Right-click → **3D Object → Plane** (Ground)
2. Right-click → **3D Object → Cube** (Object to place)
3. Rename cube as **PlaceableObject**

Step 3: Create Placement Indicator

1. Right-click → **3D Object → Cylinder / Quad**
2. Rename it as **PlacementIndicator**
3. Adjust:
    - Scale → make it small and flat
    - Position → keep slightly above ground
4. Change material color (e.g., green) for visibility

Step 4: Create Script

1. In Project → Right-click → **Create → C# Script**
2. Name it:

PlacementIndicator

3. Open the script

Step 5: Write the Code

```
using UnityEngine;

public class PlacementIndicator : MonoBehaviour
{
   public GameObject indicator;
   public Camera cam;

   void Update()
   {
      Ray ray = cam.ScreenPointToRay(Input.mousePosition);
      RaycastHit hit;
```

```
        if (Physics.Raycast(ray, out hit))
        {
            indicator.SetActive(true);
            indicator.transform.position = hit.point;
        }
        else
        {
            indicator.SetActive(false);
        }
    }
}
```

Step 6: Attach Script

1. Create empty object → rename **PlacementManager**
2. Attach **PlacementIndicator script**
3. Assign:
    - Indicator → PlacementIndicator object
    - Camera → Main Camera

Step 7: Add Object Placement Logic (Optional)

Modify script to place object on click:

```
if (Input.GetMouseButtonDown(0))
{
    Instantiate(Resources.Load("PlaceableObject"), hit.point, Quaternion.identity);
}
```

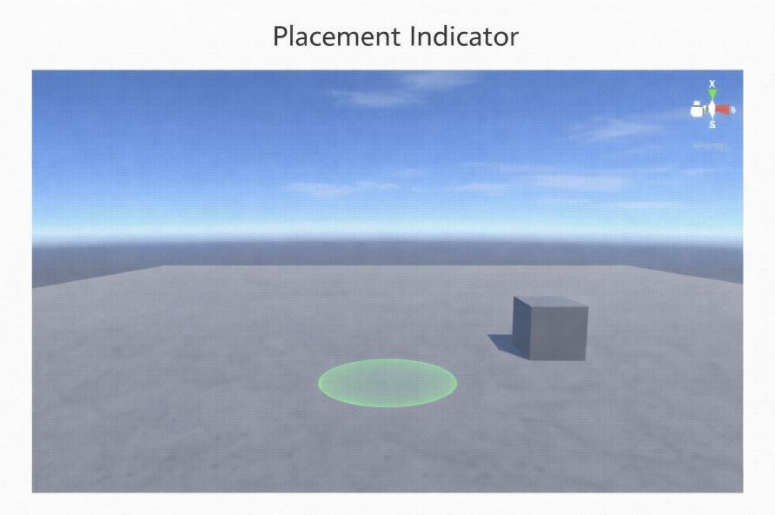
Step 8: Save Scene

1. Click **File → Save As**
2. Name it (e.g., **PlacementScene**)

Step 9: Run the Scene

1. Click **Play**
2. Move mouse:
    - Indicator follows surface
3. Click:
    - Object gets placed

Output



Result

Thus, a placement indicator system was successfully created and scripted in Unity.

EXERCISE 7

DATE

Scene Creation with a Controllable Character

Aim

To create a simple scene with a controllable character (cube) that can move forward, backward, left, and right using keyboard input.

Procedure

Step 1: Create a New Project

1. Open Unity Hub
2. Click **New Project**
3. Select **3D Core Template**
4. Click **Create**

Step 2: Create the Scene

1. In **Hierarchy**, right-click → **3D Object** → **Plane** (Ground)
2. Right-click → **3D Object** → **Cube** (Player)
3. Rename the cube as **Player**

Step 3: Adjust Camera

1. Select **Main Camera**
2. Position it so you can clearly see the cube
    - Example: Position → (0, 5, -10)
    - Rotation → (30, 0, 0)

Step 6: Create Movement Script

1. In Project panel → Right-click → **Create** → **C# Script**
2. Name it:
   PlayerMovement
3. Double-click to open the script

Step 7: Write the Code

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class playermovement : MonoBehaviour
{
  Vector3 movement;
  public float speed;
```

```
    // Start is called before the first frame update


    // Update is called once per frame
    private void Update()
    {
     movement.x = Input.GetAxis("Horizontal")*speed*Time.deltaTime;
     movement.z = Input.GetAxis("Vertical") *speed*Time.deltaTime;
     transform.Translate(movement,Space.Self);
    }
}
```

Step 8: Attach Script to Cube

1. Select **Player (Cube)**
2. Click **Add Component**
3. Attach **PlayerMovement script**

Step 9: Save the Scene

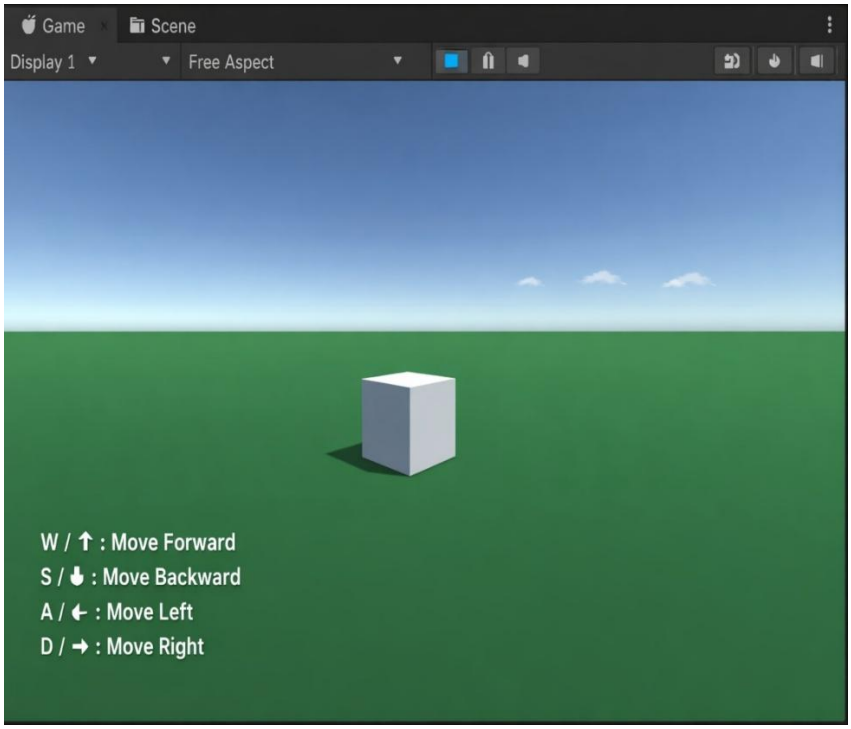
1. Click **File → Save As**
2. Name it (e.g., **MovementScene**)

Step 10: Run the Scene

1. Click **Play Button**
2. Use keyboard:
    - **W / Up Arrow** → Move Forward
    - **S / Down Arrow** → Move Backward
    - **A / Left Arrow** → Move Left
    - **D / Right Arrow** → Move Right

The cube moves smoothly in all four directions based on keyboard input.

Output



Result

The simple scene with a controllable cube was successfully created, and the cube moved in all directions using keyboard inputs.

EXERCISE 8

DATE

User Interaction With Objects (Pick up & Drop)

Aim

To create a script that allows the player to interact with objects in the scene, such as picking up, carrying, and dropping objects using keyboard input.

Procedure

Step 1: Create the Scene Environment

1. In the **Hierarchy Window**, right-click → **3D Object** → **Plane** (this will act as the ground).
2. Right-click → **3D Object** → **Cube** (this will be the pickup object).

Step 2: Add Rigidbody and Tags

1. Select the **pickup cube**.
2. In the **Inspector**, click **Add Component** → **Rigidbody**.
3. At the top of the Inspector, click **Tag** → **Add Tag**.
4. Create a new tag called **Pickup**.
5. Assign the **Pickup** tag to the cube.

Step 4: Create Hold Position

1. Select **Main Camera** in the Hierarchy.
2. Right-click → **Create Empty**.
3. Rename it **HoldPoint**.
4. Move the HoldPoint slightly **in front of the camera**.

Step 5: Create Interaction Script

1. In the **Project Window**, right-click → **Create** → **C# Script**.
2. Rename the script **PlayerInteraction**.
3. Double-click the script to open it in the code editor.
4. Replace the default code with the interaction script provided.

```
using UnityEngine;
public class PlayerInteraction : MonoBehaviour
{
    public float interactDistance = 3f;
```

```
  public Transform holdPosition;
private GameObject heldObject;
void Update()
{
        if (Input.GetKeyDown(KeyCode.E))
        {
                if (heldObject == null)
                {
                PickUpObject();
                }
                else
                 {
                DropObject();
                }
        }
}
 void PickUpObject()
{
        RaycastHit hit;
  if (Physics.Raycast(transform.position, transform.forward, out hit, interactDistance))
 {
          if (hit.collider.CompareTag("Pickup"))
        {
                heldObject = hit.collider.gameObject':
                heldObject.GetComponent<Rigidbody>().useGravity = false;
                heldObject.GetComponent<Rigidbody>().isKinematic = true;
                heldObject.transform.position = holdPosition.position;
                heldObject.transform.parent = holdPosition;
           }

 }
}
void DropObject()
```

```
{
        heldObject.GetComponent<Rigidbody>().useGravity = true;
        heldObject.GetComponent<Rigidbody>().isKinematic = false;
       heldObject.transform.parent = null;
       heldObject = null;
 }
 }
```

Step 6: Attach the Script

1. Select the **Main Camera**.
2. Drag the **PlayerInteraction** script onto the camera.
3. In the Inspector, locate the **Hold Position** field.
4. Drag the **HoldPoint** object into this field.

Step 7: Arrange Objects

Cube Position
X = 0
Y = 1
Z = 3

Camera Position

X = 0
Y = 1
Z = 0

HoldPoint position:

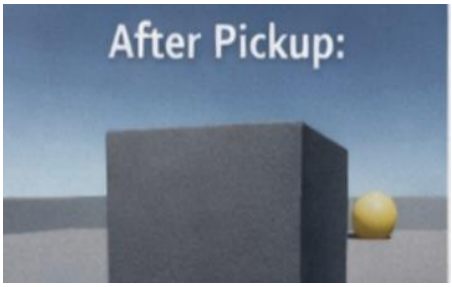
X = 0
Y = 0
Z = 1

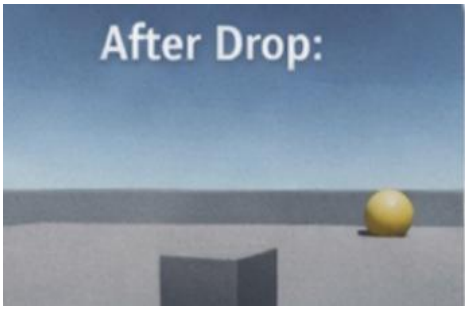
Step 8: Run the Scene

1. Click the **Play** button at the top of the Unity Editor.
2. Look at the pickup object and press **E**.
3. The object will **attach to the HoldPoint**.
4. Press **E again** to **drop the object**.
5. Look at the trigger object and press **E** to **activate the event**.

Output







Result

Thus, the player interaction system for picking up and dropping objects was successfully implemented in Unity.

EXERCISE 9

DATE

Rotate an Object Around its Axis

Aim:
To develop a script in Unity that rotates an object (e.g., a sphere) around its own axis when the player presses specific keys such as Q and E.

Procedure

Step 1: Open Unity Hub

1. Launch **Unity Hub**.
2. Click **New Project**.
3. Select **3D Core Template**.
4. Enter the project name **ObjectRotation**.
5. Click **Create Project**.

Step 2: Create a Sphere Object

1. In the **Hierarchy window**, right-click.
2. Select **3D Object → Sphere**.
3. A sphere will appear in the **Scene View**.

Step 3: Reset the Sphere Position

1. Select the **Sphere** in the Hierarchy.
2. In the **Inspector window**, go to **Transform**.

   Set the values:

   - Position **X = 0**
   - Position **Y = 0**
   - Position **Z = 0**

Step 4: Create a C# Script

1. In the **Project window**, right-click.
2. Select **Create → C# Script**.
3. Name the script **RotateObject**.

Step 5: Open the Script

1. Double-click the **RotateObject** script.
2. It will open in **Visual Studio / VS Code**.

```
using UnityEngine;
public class RotateObject : MonoBehaviour
{
    public float rotationSpeed = 100f;
    void Update()
  {
    if (Input.GetKey(KeyCode.Q))
    {
      transform.Rotate(Vector3.up * -rotationSpeed * Time.deltaTime);
    }
    if (Input.GetKey(KeyCode.E))
    {
      transform.Rotate(Vector3.up * rotationSpeed * Time.deltaTime);
    }
  }
}
```

Step 7: Attach Script to the Sphere

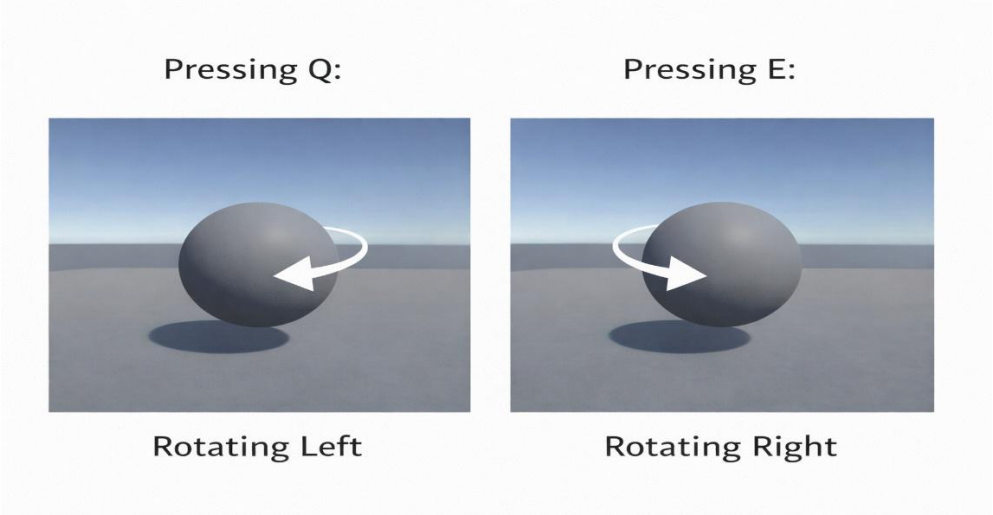
1. Go back to **Unity Editor**.
2. Select the **Sphere** object.
3. Drag the **RotateObject script** from the **Project window** to the **Inspector panel** of the Sphere.

Step 8: Run the Scene

1. Click the **Play button** at the top of the Unity Editor.
2. Press the keys:
    - **Q** → Sphere rotates **left**
    - **E** → Sphere rotates **right**

Output

The **Sphere rotates around its axis** when the player presses **Q and E keys**.



Result

Thus, a script to rotate an object using keyboard input was successfully implemented in Unity..

EXERCISE 10

DATE

2D Coin Animation Using Sprites

Aim

To create a coin twisting (flip) animation using sprites in Unity by implementing frame-by-frame animation techniques

Procedure

Step 1: Create a New Project

1. Open Unity Hub
2. Click **New Project**
3. Select **2D Core Template**
4. Click **Create**

Step 2: Import Coin Sprites

1. Download or collect **coin sprite images** (multiple frames of rotation)
2. In Unity → Go to **Assets**
3. Drag and drop all sprite images into Assets

Step 3: Set Sprite Mode

1. Select all imported sprites
2. In Inspector:
    - Texture Type → **Sprite (2D and UI)**
    - Sprite Mode → **Multiple** (if spritesheet)
3. Click **Apply**

Step 4: Slice Sprites (if needed)

1. Select sprite sheet
2. Click **Sprite Editor**
3. Click **Slice → Automatic/Grid**
4. Click **Apply**

Step 5: Create Animation

1. Select all coin sprite frames (in order)
2. Drag them into the **Scene**
3. Unity will prompt:
    - Save animation → Name it **CoinTwist**
4. Unity automatically creates:
    - Animation clip
    - Animator Controller

Step 6: Adjust Animation Speed

1. Select **CoinTwist animation**
2. In Inspector:
    - Set **Samples (FPS)** → e.g., 10–15
3. Enable **Loop Time**

Step 7: Create Coin Object

1. In Hierarchy → Create empty object → rename **Coin**
2. Add **Sprite Renderer**
3. Assign first coin sprite
4. Add **Animator component**
5. Assign **Animator Controller (CoinTwist)**

Step 8: Create Script (Optional Control)

1. Create C# Script → name: CoinSpin
2. Add code:

```
using UnityEngine;

public class CoinSpin : MonoBehaviour
{
   public Animator animator;

   void Update()
   {
      if (Input.GetKeyDown(KeyCode.Space))
      {
         animator.Play("CoinTwist");
      }
   }
}
```

Step 9: Attach Script

1. Select **Coin object**
2. Attach **CoinSpin script**
3. Assign Animator in Inspector

Step 10: Run the Scene

1. Click **Play**
2. Press **Space key**
3. Coin will **twist/flip animation**

Output



Result

Thus, a coin twist animation using sprites was successfully implemented in Unity.

EXERCISE 11

DATE

Implementation to Show Portal Planets

Aim

To implement a portal-based planet viewing application in Unity, where the user can view and explore different planets through a virtual portal in an AR/VR environment.

Procedure

Step 1: Create a Unity Project

1. Open **Unity Hub**.
2. Click **New Project**.
3. Select the **3D Core** template.
4. Enter the project name as **Portal Planets**.
5. Click **Create Project**.

Step 2: Create the Planet

1. Go to **GameObject → 3D Object → Sphere**.
2. Rename the sphere as **Planet**.
3. Apply a suitable **planet texture** to the sphere.
4. Adjust the size and position of the planet.
5. Add a light source to provide proper illumination.

Step 3: Create the Portal

1. Create a **Cylinder, Circle, or Torus** to form the portal frame.
2. Apply a suitable material to the portal.
3. Add a glowing or animated effect to the portal.
4. Position the portal in front of the user.

Step 4: Create the Portal View

1. Create a separate camera for viewing the planet.
2. Position the camera to look towards the planet.
3. Render the camera view through the portal.
4. Configure the portal so that the planet appears when the user looks through it.

Step 5: Add Multiple Planets

1. Create additional spheres for different planets.
2. Apply suitable textures to each planet.
3. Place the planets in separate portal scenes or locations.

4. Add labels or information panels for the planets if required.

Step 6: Configure VR Interaction

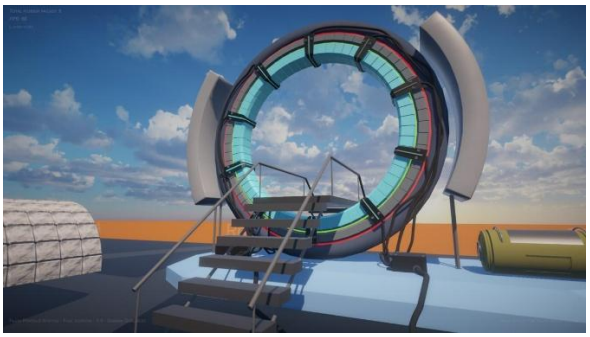
1. Add an **XR Origin** to the scene.
2. Configure the VR camera and controllers.
3. Allow the user to approach or look through the portal.
4. Test the portal interaction using the VR controller.

Step 7: Test the Application

1. Save the scene.
2. Click **Play** in Unity.
3. View the portal through the VR environment.
4. Observe the planet displayed through the portal.
5. Test the interaction with different planets.

Output

A virtual portal is displayed in the VR environment. When the user views the portal, a planet appears through the portal, creating the effect of looking into another planetary environment.



Result

The Portal Planets application was successfully implemented in Unity.